Bags, Vectors & Transformers

Contextual Embeddings & Transformers · Day 3

Denise J. Roth, PhD Candidate
Strategic Communication Group · Wageningen University & Research

Where We Have Been

Two days, one journey — from counting words to representing meaning:

Day 1: Bags

· Words as counts

· Document-term matrix

· TF-IDF, dictionaries, topics

· Simple, transparent

Day 2: Vectors

· Meaning from context

· Word2Vec, GloVe

· Dense embeddings

· **But: one vector per word**

Day 3: Transformers

· Meaning from *this* context

· Attention

· BERT & friends

· **A vector per use**

Today we close the loop: the context problem that static embeddings could not solve.

Warm-up: Which Bank Is Which?

YOUR TURN Sort these six sentences into two groups. Call out the numbers.

1. I sat on the grassy **bank** of the river.
2. The **bank** refused to give us a mortgage.
3. We fished from the muddy **bank** of the stream.
4. She works as an analyst at a large investment **bank**.
5. The flood washed away part of the river **bank**.
6. I deposited my paycheck at the **bank**.

Warm-up: Which Bank Is Which?

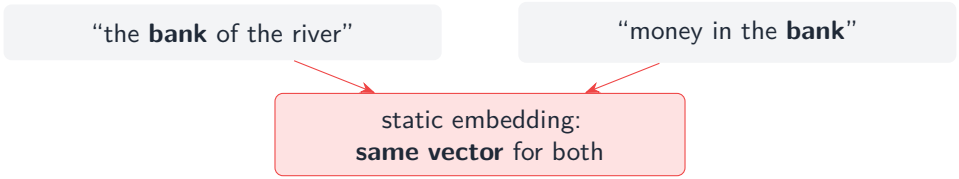
YOUR TURN Sort these six sentences into two groups. Call out the numbers.

1. I sat on the grassy **bank** of the river.
2. The **bank** refused to give us a mortgage.
3. We fished from the muddy **bank** of the stream.
4. She works as an analyst at a large investment **bank**.
5. The flood washed away part of the river **bank**.
6. I deposited my paycheck at the **bank**.

Water: 1, 3, 5. Money: 2, 4, 6. Easy for you, because you read the whole sentence. A static embedding like GloVe gives **bank** the same vector in all six. In the notebook (Section 6) you'll test whether BERT sorts these exact sentences the way you did.

The Problem, One More Time

Static embeddings give each word **one fixed vector**. But words change meaning with context — and a single vector cannot be in two places at once.



We want representations that **read the sentence** and give "bank" a *different* vector depending on whether it is about water or money. That is what **contextual embeddings** do.

What Contextual Embeddings Unlock

New capabilities

· **Word sense** disambiguation (“bank”, “spring”)

· Represent **whole sentences** richly

· Capture **syntax & negation** (“not good” $\neq$ “good”)

· Strong **transfer**: fine-tune for your task with little data

What this means for you

· State-of-the-art **classification**

· Better **sentiment / stance** on hard cases

· **Semantic search** across documents

· A foundation for the **LLMs** everyone is talking about

The jump from static to contextual is what took NLP from “useful” to “everywhere” over the last few years.

The Breakthrough: “Attention Is All You Need”

“Attention Is All You Need”

Vaswani et al., 2017 — the paper that introduced the **transformer**

Before 2017, models read text **word by word, in order** (recurrent networks) — slow, and forgetful over long distances. The transformer threw that out and replaced it with one mechanism: **attention**, which lets every word look at every other word **at once**.

That single idea powers **BERT**, **GPT**, and essentially every modern language model. The next few slides build up what “attention” actually means.

What Does *it* Refer To?

YOUR TURN Hands up: does **it** mean the animal, or the street?

1. The animal didn't cross the street because **it** was too **tired**.

What Does *it* Refer To?

YOUR TURN Hands up: does **it** mean the animal, or the street?

1. The animal didn't cross the street because **it** was too **tired**.
   → **it** = the animal

What Does *it* Refer To?

YOUR TURN Hands up: does **it** mean the animal, or the street?

1. The animal didn't cross the street because **it** was too **tired**.
   → **it** = the animal

2. The animal didn't cross the street because **it** was too **wide**.

What Does *it* Refer To?

YOUR TURN Hands up: does **it** mean the animal, or the street?

1. The animal didn't cross the street because **it** was too **tired**.
   → **it** = the animal

2. The animal didn't cross the street because **it** was too **wide**.
   → **it** = the street

One word at the very end changed, and **it** now points to something else. To get this right, a model has to connect **it** with *tired* or *wide*, several words away. Learning those connections is what **attention** does. (Want to see where BERT's *it* looks? Notebook Section 7.)

Attention: The Core Intuition

To understand a word, you look at the **other words that give it meaning**. Attention makes a model do exactly this — for every word, it decides **how much to focus on each other word** in the sentence.



To represent **"it"**, the model pays **strong attention** to **"animal"** (that is what "it" refers to) and a little to "tired". The thickness of each arrow is the **attention weight** — how relevant that word is.

Crucially, the model learns these weights from data — nobody hand-codes that "it" means "animal" here.

How Attention Works — Without the Math

For each word, attention runs a little **matching process**. A useful analogy is **searching**:

Query

What *this* word is looking for.

"**it**": what do I refer to?

Key

What each *other* word offers.

"animal": I'm a noun, a thing

Value

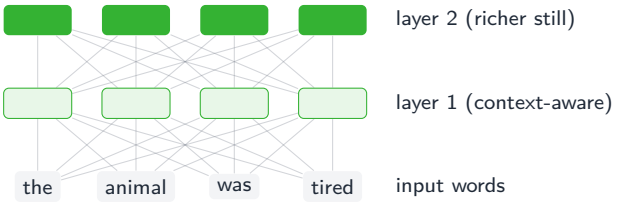
The information actually *passed along* when there's a match.

The word's **query** is compared with every other word's **key**. Good matches get high **attention weights**, and the matching words' **values** are blended together to form the word's new, **context-aware** representation.

Query–key–value: the whole transformer is built by stacking this one operation many times.

From One Layer to a Deep Model

Attention runs **in parallel** for every word (this is **self-attention** — the sentence attends to itself). Then we **stack many layers**, each refining the representations further.



Each layer lets every word gather information from all the others. After a dozen layers, each word's vector reflects the **entire sentence**. **BERT** has 12 such layers; large models have many more.

How BERT Learns: Masked Language Modeling

A transformer needs to be **trained** on something. **BERT** — *Bidirectional Encoder Representations from Transformers* (Devlin et al., 2018) — uses a clever self-supervised task: **fill in the blank**.

"the animal didn't cross because **[MASK]** was tired"

↓

predict the hidden word: **"it"**

Hide 15% of words and train the model to **predict them from context**. To do this well, it *must* learn grammar, meaning, and world knowledge — all **without human labels**.

The **"Bidirectional"** in the name matters here: to guess the blank, BERT reads the words *both before and after* it — unlike left-to-right models that only look back. *(The same "fake task" trick as Word2Vec, but far more powerful.)*

Be BERT: Fill in the Blank

YOUR TURN Call out the word you'd put in each gap. Then we ask DistilBERT, live.

1. The parliament voted to `[MASK]` the new law.

Be BERT: Fill in the Blank

YOUR TURN Call out the word you'd put in each gap. Then we ask DistilBERT, live.

1. The parliament voted to `[MASK]` the new law.
2. Wageningen is a `[MASK]` in the Netherlands.

Be BERT: Fill in the Blank

YOUR TURN Call out the word you'd put in each gap. Then we ask DistilBERT, live.

1. The parliament voted to `[MASK]` the new law.
2. Wageningen is a `[MASK]` in the Netherlands.
3. The farmers protested against the new `[MASK]` regulations.

Be BERT: Fill in the Blank

YOUR TURN Call out the word you'd put in each gap. Then we ask DistilBERT, live.

1. The parliament voted to `[MASK]` the new law.
2. Wageningen is a `[MASK]` in the Netherlands.
3. The farmers protested against the new `[MASK]` regulations.
4. The central bank raised interest `[MASK]` to fight inflation.

Be BERT: Fill in the Blank

YOUR TURN Call out the word you'd put in each gap. Then we ask DistilBERT, live.

1. The parliament voted to `[MASK]` the new law.
2. Wageningen is a `[MASK]` in the Netherlands.
3. The farmers protested against the new `[MASK]` regulations.
4. The central bank raised interest `[MASK]` to fight inflation.
5. The nurse said that `[MASK]` would be late. And the engineer?

Be BERT: Fill in the Blank

YOUR TURN Call out the word you'd put in each gap. Then we ask DistilBERT, live.

1. The parliament voted to `[MASK]` the new law.
2. Wageningen is a `[MASK]` in the Netherlands.
3. The farmers protested against the new `[MASK]` regulations.
4. The central bank raised interest `[MASK]` to fight inflation.
5. The nurse said that `[MASK]` would be late. And the engineer?

You just did BERT's training task: fill in the blank using the words on **both** sides. BERT did this over and over, on billions of words. **Notebook Section 4** asks the model the same questions, and number 5 turns into a quick bias probe.

What BERT Read

Pre-training needs no labels, only **a lot of text**: the text itself is the training signal, because every sentence can be turned into fill-in-the-blank questions.

BooksCorpus

About **800 million words** from 11,038 unpublished books, collected online.

English Wikipedia

About **2.5 billion words**: running text only, without lists, tables and headers.

Together about **3.3 billion words**. Reading eight hours a day at 250 words a minute, that's about **75 years** of reading. Later models read far more: RoBERTa (2019) used **160 GB** of text, roughly ten times as much, adding news and web pages.

What's in the data ends up in the model. A later audit of BookCorpus found thousands of duplicate books, skewed genres, and likely copyright violations (Bandy & Vincent, 2021). Wikipedia has its own gaps in who and what gets written about. This is where associations like the *nurse*/*she* pattern come from.

What Pre-training Costs

model	parameters	hardware	time
BERT-base (2018)	110 million	16 TPU chips	4 days
BERT-large (2018)	340 million	64 TPU chips	4 days
DistilBERT (2019)	66 million	8 V100 GPUs	about 90 hours

In money and CO_2

One estimate for BERT-base on 64 GPUs: 79 hours, **$3,751 to $12,571** of cloud computing, and about **650 kg of CO_2**, which the authors compare to a flight across the US (Strubell et al., 2019).

For you

Fine-tuning DistilBERT on the bill titles takes **minutes** on one free Colab GPU. That's why almost everyone downloads a pre-trained model instead of training one.

It has become cheaper since: one day on one consumer GPU now gets close to BERT-base (Geiping & Goldstein, 2023).

Two Families: Encoders and Decoders

Encoders (BERT-style)

- Read the **whole sentence at once**
- Built for **understanding** & analysis
- Great for **classification**, search, feature extraction
- **Our focus** — best fit for social-science analysis

Decoders (GPT-style)

- Read **left to right**, predict the next word
- Built for **generation**
- Power **ChatGPT** and the LLMs

Same attention engine, different objective.

For measuring and classifying text — what social scientists usually want — encoders like BERT are typically the right tool.

Why Word Pieces?

A model needs a **fixed vocabulary**: every entry gets its own row in the model's lookup table. But which units should the entries be?

Whole words

Every name, typo, plural and new word needs an entry. Anything unseen becomes `[UNK]`: unknown.

Single characters

A tiny vocabulary, but very long sequences, and a letter on its own means almost nothing.

Word pieces

About **30,000** pieces. Common words stay whole; rarer words are built from smaller pieces. Almost nothing is unknown.

How the vocabulary is built (BERT's method is called *WordPiece*), once, before pre-training:

1. Start with single characters.
2. Merge the two pieces that occur together most often in the training text: `t` + `h` → `th`, then `th` + `e` → `the`, and so on.
3. Stop at about 30,000 pieces.

How a Word Gets Cut

The rule: take the **longest piece** in the vocabulary that the word starts with, then do the same with the rest. A piece that continues a word is marked ##.

vaccination

`va` (vac, vacc, vacci aren't in the vocabulary)
`##cci`
`##nation`

vaccine and *vaccines* are single pieces; *vaccination* is three. What decides is how often a string appeared in the training text, not what it means.

The Ministry of Agriculture published new nitrogen-emission guidelines for dairy farmers.

11 words → 14 pieces

De nieuwe stikstofregels zijn een ramp voor de melkveehouders in Gelderland.

11 words → 30 pieces (English model)

Why it matters

- ✔ The 512 limit counts **pieces**, not words.
- ✔ Chopped-up words give the model less to go on.
- ✔ For Dutch, use a model with a Dutch vocabulary (BERTje, RobBERT).

How Many Pieces?

YOUR TURN Now you know the rule. Into how many pieces does DistilBERT cut each of these?

word
democracy
misinformation
mRNA
ChatGPT
#MeToo
stikstofuitstoot

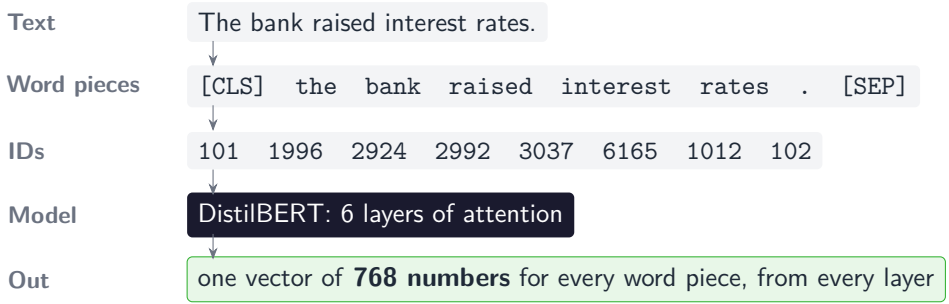
How Many Pieces?

YOUR TURN Now you know the rule. Into how many pieces does DistilBERT cut each of these?

word	pieces	how many
democracy	`democracy`	**1**
misinformation	`mis · ##in · ##form · ##ation`	**4**
mRNA	`mrna`	**1**
ChatGPT	`chat · ##gp · ##t`	**3**
#MeToo	`# · met · ##oo`	**3**
stikstofuitstoot	`st · ##ik · ##sto · ##fu · ##its · ##to · ##ot`	**7**

`##` = glued to the piece before. Everything is lowercased first (the *uncased* model). Common English words stay whole; names, new words and Dutch fall apart, which leaves the model less to go on. More in **notebook Section 3**.

What Goes In, What Comes Out



- `[CLS]` and `[SEP]` are **special tokens** added to every input.
- One vector for a whole text: take the `[CLS]` vector, or **average** all the word-piece vectors.
- At most **512 word pieces** go in at once. Longer texts are cut off or split.

Mechanics: The Takeaways

- ✔ **Contextual embeddings** give a word a **different vector in each sentence** — solving the "bank" problem
- ✔ They are built on the **transformer** ("Attention Is All You Need", 2017)
- ✔ **Attention** lets each word focus on the other words that matter — via **query, key, value**
- ✔ **BERT** learns by **masked language modeling**: fill in the blank, no labels needed
- ✔ Pre-training takes **billions of words** and serious hardware, so you **download** the result
- ✔ **Encoders** (BERT) are for understanding; **decoders** (GPT) are for generation
- ✔ BERT reads **word pieces** and returns **one vector per piece**, in every layer

Next: notebook Part A. We open BERT up: word pieces, what comes out of the model, the "bank" test, and your fill-in-the-blank guesses.